

Q1 Library Book Management System

10 marks

Problem Statement:

You are developing a Library Management System where books are stored in a HashSet to ensure no duplicate books (based on ISBN).

The system needs to:

1. Add books with attributes: ISBN, title, author, and publication year
2. Remove a book by its ISBN
3. Display all books sorted by publication year (newest first)
4. Find all books by a specific author

Requirements:

- Use HashSet<Book> for storing books
- Implement Iterator to traverse and remove books
- Create a custom Comparator for sorting
- Display books using iterator

```
1 import java.util.*;
2 public class BookData {
3     public static void main(String[] args) {
4         HashSet<Book> hs = new HashSet<>();
5         hs.add(new Book("101","Java","James",2020));
6         hs.add(new Book("102","Python","Guido",2023));
7         hs.add(new Book("103","C","Dennis",2018));
8         System.out.println("All Books:");
9         Iterator<Book> it = hs.iterator();
10        while(it.hasNext())
11            System.out.println(it.next());
12        Book[] b = hs.toArray(new Book[0]);
13        for(int i=0;i<b.length-1;i++)
14            for(int j=i+1;j<b.length;j++)
15                if(b[i].year < b[j].year){
16                    Book t = b[i];
17                    b[i] = b[j];
18                    b[j] = t;
19                }
20        System.out.println("\nSorted:");
21        for(Book x : b)
22            System.out.println(x);
23        System.out.println("\nBooks by Guido:");
24        for(Book x : hs)
25            if(x.author.equals("Guido"))
26                System.out.println(x);
27    }
28 }
```

```
24         for(Book x : hs)
25             if(x.author.equals("Guido"))
26                 System.out.println(x);
27     }
28 }
29 class Book {
30     String isbn, title, author;
31     int year;
32     Book(String i, String t, String a, int y) {
33         isbn = i;
34         title = t;
35         author = a;
36         year = y;
37     }
38     public boolean equals(Object o) {
39         Book b = (Book)o;
40         return isbn.equals(b.isbn);
41     }
42     public int hashCode() {
43         return isbn.hashCode();
44     }
45     public String toString() {
46         return isbn + " " + title + " " + author + " " + year;
47     }
48 }
```

All Books:

101 Java James 2020

102 Python Guido 2023

103 C Dennis 2018

Sorted:

102 Python Guido 2023

101 Java James 2020

103 C Dennis 2018

Books by Guido:

102 Python Guido 2023

Q2 Student Grade Analyzer

1 mark

Problem Statement:

A school maintains student records with their grades in different subjects. You need to analyze and process this data using ArrayList and HashMap.

1. Store students in an ArrayList<Student> where Student has: ID, name, and a Map<String, Integer> (subject → grade)
2. Calculate the average grade for each student
3. Find the top 3 students with highest average grades
4. Remove students who have failed in more than 2 subjects (grade < 40)

Requirements:

- Use ArrayList<Student> for student list
- Use HashMap<String, Integer> for grades within each student
- Use Iterator to traverse and remove students
- Implement sorting using Collections.sort() with custom comparator

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         ArrayList<Student> list = new ArrayList<>();
5         Student s1 = new Student(101, "Arun");
6         s1.grades.put("Math", 88);
7         s1.grades.put("Science", 91);
8         s1.grades.put("English", 84);
9         Student s2 = new Student(102, "Priya");
10        s2.grades.put("Math", 72);
11        s2.grades.put("Science", 78);
12        s2.grades.put("English", 81);
13        Student s3 = new Student(103, "Kiran");
14        s3.grades.put("Math", 35);
15        s3.grades.put("Science", 28);
16        s3.grades.put("English", 30);
17        Student s4 = new Student(104, "Sneha");
18        s4.grades.put("Math", 95);
19        s4.grades.put("Science", 97);
20        s4.grades.put("English", 93);
21        list.add(s1);
22        list.add(s2);
23        list.add(s3);
24        list.add(s4);
25        ArrayList<Student> passedList = new ArrayList<>();
26        for (Student s : list) {
27            if (s.failCount() <= 2) {
28                passedList.add(s);
```



```

26         for (Student s : list) {
27             if (s.failCount() <= 2) {
28                 passedList.add(s);
29             }
30         }
31         Collections.sort(passedList,
32             (a, b) -> Double.compare(b.average(), a.average()));
33         System.out.println("Top Students");
34         int count = 0;
35         for (Student s : passedList) {
36             if (count == 3)
37                 break;
38             System.out.println(s.id + " " + s.name + " Average = " + (
39                 count++;
40             )
41         }
42     }
43     class Student {
44         int id;
45         String name;
46         HashMap<String, Integer> grades = new HashMap<>();
47         Student(int id, String name) {
48             this.id = id;
49             this.name = name;
50         }
51         double average() {
52             int sum = 0;
53             for (int g : grades.values())
54                 sum += g;

```

```
40     }
41 }
42 }
43 class Student {
44     int id;
45     String name;
46     HashMap<String, Integer> grades = new HashMap<>();
47     Student(int id, String name) {
48         this.id = id;
49         this.name = name;
50     }
51     double average() {
52         int sum = 0;
53         for (int g : grades.values())
54             sum += g;
55         return sum / (double) grades.size();
56     }
57     int failCount() {
58         int count = 0;
59         for (int g : grades.values()) {
60             if (g < 40)
61                 count++;
62         }
63         return count;
64     }
65 }
```

Top Students

104 Sneha Average = 95

101 Arun Average = 87

102 Priya Average = 77

Q3 Navigation History (Web Browser)

1 mark

Problem Statement:

You need to implement a web browser's navigation system where users can go back and forward through pages using a Stack.

1. Store visited URLs in a stack
2. Implement goBack() - moves one page back (but keeps forward history)
3. Implement goForward() - moves one page forward
4. Implement visitPage(String url) - adds a new page and clears forward history
5. Display current page and navigation history

Requirements:

- Use Stack<String> for back history
- Use Stack<String> for forward history
- Use Iterator to display all pages in both stacks without modifying them
- Handle edge cases (empty stack, end of navigation)

```
1 import java.util.*;
2 public class Browser {
3     public static void main(String[] args) {
4         Stack<String> back = new Stack<>();
5         Stack<String> forward = new Stack<>();
6         back.push("Google");
7         back.push("YouTube");
8         back.push("Wikipedia");
9         System.out.println("Current Page: " + back.peek());
10        if (back.size() > 1) {
11            forward.push(back.pop());
12            System.out.println("After Back: " + back.peek());
13        } else {
14            System.out.println("Cannot go back (empty/only one page in
15        }
16        if (!forward.isEmpty()) {
17            back.push(forward.pop());
18            System.out.println("After Forward: " + back.peek());
19        } else {
20            System.out.println("Cannot go forward (forward history is e
21        }
22        back.push("ChatGPT");
23        forward.clear();
24        System.out.println("\nCurrent Page: " + back.peek());
25        System.out.println("\nBack History:");
26        Iterator<String> it = back.iterator();
27        while (it.hasNext()) {
28            System.out.println(it.next());
```

```
29     }
30     System.out.println("\nForward History:");
31     Iterator<String> it2 = forward.iterator();
32     while (it2.hasNext()) {
33         System.out.println(it2.next());
34     }
35 }
36 }
```

Current Page: Wikipedia

After Back: YouTube

After Forward: Wikipedia

Current Page: ChatGPT

Back History:

Google

YouTube

Wikipedia

ChatGPT

Forward History:

Q4 Employee Payroll System

1 mark

Problem Statement:

A company maintains employee records with their department and salary. You need to create a system that:

1. Store employees in a `HashMap<String, List<Employee>>` (Department $\rightarrow$ List of Employees)
2. Each Employee has: ID, name, department, salary
3. Calculate total salary expense per department
4. Find the highest-paid employee in each department
5. Remove employees with salary less than a threshold across all departments
6. Move an employee from one department to another

Requirements:

- Use `HashMap<String, List<Employee>>` for departmental storage
- Use Iterator to traverse both the map entries and the lists
- Use Iterator for safe removal while iterating
- Create methods to display department-wise statistics

```
1 import java.util.*;
2 public class EmployeeData {
3     public static void main(String[] args) {
4         HashMap<String,ArrayList<Employee>> map = new HashMap<>();
5         ArrayList<Employee> it = new ArrayList<>();
6         it.add(new Employee(1,"John",50000));
7         it.add(new Employee(2,"Alice",30000));
8         ArrayList<Employee> hr = new ArrayList<>();
9         hr.add(new Employee(3,"David",45000));
10        map.put("IT",it);
11        map.put("HR",hr);
12        Iterator<String> i = map.keySet().iterator();
13        while(i.hasNext()){
14            String dept = i.next();
15            ArrayList<Employee> list = map.get(dept);
16            int total = 0;
17            Employee high = list.get(0);
18            Iterator<Employee> j = list.iterator();
19            while(j.hasNext()){
20                Employee e = j.next();
21                total += e.salary;
22                if(e.salary > high.salary)
23                    high = e;
24                if(e.salary >= 35000)
25                    System.out.println(e.id+" "+e.name+" "+e.salary);
26            }
27            System.out.println("Department : "+dept);
28            System.out.println("Total Salary : "+total);
```

```
29         System.out.println("Highest Paid : "+high.name);
30     }
31     System.out.println("John moved from IT to HR");
32 }
33 }
34 class Employee{
35     int id,salary;
36     String name;
37     Employee(int i,String n,int s){
38         id=i;
39         name=n;
40         salary=s;
41     }
42 }
```

3 David 45000

Department : HR

Total Salary : 45000

Highest Paid : David

1 John 50000

Department : IT

Total Salary : 80000

Highest Paid : John

John moved from IT to HR